1. DATASET DESCRIPTION

For this assignment, I selected the standard **Telco Customer Churn** dataset available publicly on Kaggle. This dataset provides customer level metrics for a telecommunications provider, making it ideal for practicing real-world data hygiene, type parsing, missing value treatment, and feature transformations without artificial constraints.

Property	Dataset Attribute Details
Dataset Name	Telco Customer Churn (Kaggle Dataset)
Dimensions	7,043 rows and 21 columns
Target Feature	`Churn` (Binary: 'Yes' or 'No')
Main Predictors	Demographics (`gender`, `SeniorCitizen`), Services (`InternetService`, `OnlineSecurity`), and Account details (`tenure`, `Contract`, `MonthlyCharges`, `TotalCharges`).
Primary Objective	Analyze customer attrition metrics and prepare a clean machine learning-ready pipeline to predict potential customer churn.

2. DATA CLEANING: PHASE 1 — DIAGNOSTICS & TYPE CORRECTION

Upon initial inspection, the dataset revealed explicit data type mismatches where numeric features were read as generic text strings, along with implicit missing values stored as spaces.

```
# Importing required libraries and loading data import pandas as pd import numpy as np df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churn.csv') # Inspecting structural types and non-null counts print(df.info()) print("\nUnique values in TotalCharges check:", (df['TotalCharges'] == " ").sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
... TotalCharges 7043 non-null object
dtypes: float64(1), int64(2), object(18)
Unique values in TotalCharges check: 11
```

```
# 1. Correcting inappropriate data types & handling hidden missing values # TotalCharges was read as object due to 11 empty space string values (" ") df['TotalCharges'] = df['TotalCharges'].replace(" ", np.nan) df['TotalCharges'] = pd.to_numeric(df['TotalCharges']) # Verifying new missing values generated after explicit casting print("Missing TotalCharges count:", df['TotalCharges'].isnull().sum())
```

```
Missing TotalCharges count: 11
```

Cleaning Decision & Justification: The column `TotalCharges` was stored as a string object because 11 rows contained whitespace characters `" "`. Converting these to NaN and casting to float correctly reflects numerical continuity. Further inspection showed these 11 rows had `tenure = 0` (brand-new accounts), meaning zero accumulated charges.

2. DATA CLEANING: PHASE 2 — MISSING VALUES, DUPLICATES & OUTLIERS

```
# Imputing missing TotalCharges for zero-tenure accounts df['TotalCharges'] = df['TotalCharges'].fillna(0.0) # 2. Detecting duplicate records duplicate_count = df.duplicated().sum() print(f"Total Duplicate Rows Found: {duplicate_count}") # Dropping unneeded identification columns if 'customerID' in df.columns: df.drop(columns=['customerID'], inplace=True)
```

```
Total Duplicate Rows Found: 0
```

Cleaning Decision & Justification: Imputing 0.0 for `TotalCharges` aligns logically with zero-month customer tenure, avoiding data deletion. The `customerID` key was removed because unique high-cardinality strings add no statistical predictive value and induce overfitting.

DETECTING OUTLIERS VIA IQR METHOD

We applied the Interquartile Range (IQR) technique on numeric columns to examine extreme value distributions and prevent skewing our future model parameters.

```
# Outlier Detection function using IQR num_cols = ['tenure', 'MonthlyCharges', 'TotalCharges'] for col in num_cols: Q1 = df[col].quantile(0.25) Q3 = df[col].quantile(0.75) IQR = Q3 - Q1 lower_bound = Q1 - 1.5 * IQR upper_bound = Q3 + 1.5 * IQR outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)] print(f"Outliers detected in {col}: {len(outliers)}")
```

```
Outliers detected in tenure: 0
Outliers detected in MonthlyCharges: 0
Outliers detected in TotalCharges: 0
```

Outlier Decision & Analysis: The mathematical bounds confirmed zero formal outliers. Continuous features lie within bounded physical operational ranges (e.g., tenure spanning 0–72 months, monthly fees bounded by package rates). Thus, no row removal was required.

3. FEATURE ENGINEERING & VARIABLE IDENTIFICATION

Prior to encoding and scaling, features were classified into distinct operational types. I also generated a composite feature to represent average monthly spend behavior relative to total account tenure.

```
# Feature Engineering: Creating 'AvgCostPerMonth' # Adding small constant to tenure to prevent zero division df['AvgCostPerMonth'] = df['TotalCharges'] / (df['tenure'] + 1) # Variable Classification binary_cols = ['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'PhoneService', 'PaperlessBilling'] nominal_cols = ['MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaymentMethod'] target_col = 'Churn' print("Feature Engineering complete. New dataset shape:", df.shape)
```

```
Feature Engineering complete. New dataset shape: (7043, 21)
```

4. DATA PREPROCESSING: LEAKAGE-SAFE PIPELINE

To avoid **data leakage**, all transformation models (encoders and scalers) were trained exclusively on the training subset following an 80/20 stratified split, then applied to the validation subset.

```
# 1. Train-Test Split prior to scaling & encoding to prevent leakage from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler, OneHotEncoder X = df.drop(columns=['Churn']) y = df['Churn'].map({'Yes': 1, 'No': 0}) # Target encoding X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20, random_state=42, stratify=y ) print(f"Training Set Shape: {X_train.shape}, Testing Set Shape: {X_test.shape}")
```

```
Training Set Shape: (5634, 20), Testing Set Shape: (1409, 20)
```

ENCODING CATEGORICAL FEATURES & NORMALIZATION

```
# 2. Categorical One-Hot Encoding X_train_encoded = pd.get_dummies(X_train, columns=binary_cols + nominal_cols, drop_first=True) X_test_encoded = pd.get_dummies(X_test, columns=binary_cols + nominal_cols, drop_first=True) # Aligning test set columns with train set to guarantee identical feature space X_train_encoded, X_test_encoded = X_train_encoded.align(X_test_encoded, join='left', axis=1, fill_value=0) # 3. Standardizing numerical features scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges', 'AvgCostPerMonth'] scaler = StandardScaler() # Fit ONLY on training data X_train_encoded[scale_cols] = scaler.fit_transform(X_train_encoded[scale_cols]) # Transform test data using fitted mean and standard deviation X_test_encoded[scale_cols] = scaler.transform(X_test_encoded[scale_cols]) print("Preprocessing pipeline executed successfully.") print("Final X_train dimensions:", X_train_encoded.shape)
```

```
Preprocessing pipeline executed successfully.
Final X_train dimensions: (5634, 31)
```

Data Leakage Safeguard: Fitting the `StandardScaler` exclusively on `X_train` guarantees that parameters like global mean and variance from the test set do not leak into the training process.

5. NOTEBOOK SUBMISSION DETAILS

Deliverable Requirement	Student Verification & Link Status
Environment	Google Colab / Jupyter Notebook
Documentation	Complete markdown commentary written for every cleaning decision.
Public Repository Link	https://github.com/your-username/data-cleaning-assignment
LMS Compliance	Standalone PDF exported and uploaded before target deadline.